

Guia de Desenvolvimento - Dominion Arena Tatica

Arquitetura

Backend (Node.js + Express + Socket.io)

- **server.js**: Servidor principal com rotas REST e WebSocket
- **gameLogic.js**: Logica de jogo, mecanicas, calculos
- **database.js**: Conexao PostgreSQL e schema

Frontend (Next.js + React)

- **pages/**: Paginas principais (menu, jogo)
- **components/**: Componentes React reutilizaveis
- **lib/**: Utilidades (API, Socket.io)
- **styles/**: CSS global

Fluxo de Jogo

1. Jogador cria/entra em sala
2. Socket.io conecta ao servidor
3. GameLogic cria instancia do jogo
4. Fases: roll -> buy -> combat -> end
5. Estado sincronizado via WebSocket

Adicionando Novos Eventos WebSocket

No Backend (server.js):

```
socket.on('novo-evento', ({ parametro }) => {  
  const game = games.get(socket.gameId);  
  const resultado = game.novoMetodo();  
  io.to(socket.gameId).emit('evento-respondido', resultado);  
});
```

No Frontend (GameBoard.js):

```
socket.on('evento-respondido', (data) => {  
  setGameState(data);  
});  
  
// Disparar evento  
socket.emit('novo-evento', { parametro: valor });
```

Adicionando Nova Mecanica

Exemplo: Novo tipo de carta

1. Adicionar em `dominion_cards.json`
2. Adicionar logica em `gameLogic.js`
3. Adicionar evento WebSocket em `server.js`
4. Adicionar listener no frontend
5. Criar/atualizar componentes visuais

Testes

Backend:

```
cd backend  
npm test
```

Frontend:

```
cd frontend  
npm test
```

Deploy

Docker:

```
docker-compose up --build
```

Variaveis de Ambiente

Backend (.env):

- DATABASE_URL: Conexao PostgreSQL
- PORT: Porta do servidor
- NODE_ENV: development/production
- FRONTEND_URL: URL do frontend

Frontend (.env.local):

- NEXT_PUBLIC_API_URL: URL da API
- NEXT_PUBLIC_SOCKET_URL: URL do WebSocket